Car Calculator: CSEE 4280 Final Project

Trent Jones
Student
Trent.Jones@uga.edu

Abhi Boggavarapu
Student
Abhinav.Boggavarapu@uga.edu

Table of Contents

Design Explanation

1. Problem Summary

Many novice racers and consumers who care about their daily drivers do not have an easily accessible method of checking their car's under-the-hood performance. A majority of quality OBD-II readers are relatively expensive and made for users who have an idea of what to look for on their display, leaving the regular consumer at a disadvantage. Our product seeks to minimize this gap and provide an easy-to-use, inexpensive solution to this far too common problem.

By knowing how to connect to the OBD-II port of a car and the VGA or HDMI port of a monitor, anyone will be able to use this product and gain valuable insight into their automobile's performance. The design currently supports reading of the engine's rotations per minute as well as the coolant temperature, and calculates the average value of both over the duration of a trip.

The OBD-II port connects to a UART controller that transmits PID codes and receives data that is not in the decimal form that people are used to. Then the received data is stored in the FPGA device. Once stored and formatted into a readable form, this data is passed to the board's VGA output and displayed to a monitor, graphing the instantaneous values over time, as well as an average value calculation directly beside each individual graph. This design allows all of the hardware to handle the rigorous operations necessary in a way that prevents user error from tampering with the results.

The team created a project checklist to breakdown the design and set out quantifiable goals for each key piece. Each major part of the project was given three sections to define its completion. From left to right the three sections are the commitment, goal, and stretch goal; these range from the bare minimum objective to things that are a reach and would make the design stand out. The team's design incorporates a mixture of these goals across the five sections.

The project is broken down into five major parts:

- Power Design
- UI Design
- Latency Processing
- ECU Interaction
- Product Design

Each portion plays a critical role in the overarching design and functionality of the product. This section will highlight how the current design incorporates the parts shown above.

1. Power Design

Power design is described as how the product's UART module and FPGA board are given power. As with other projects, the connected computer was responsible for programming and powering the FPGA, and the UART module was programmed by an external power source.

The current model has implemented external battery power by adjusting the pin headers on the board to run off wall power instead of through the USB. A battery pack connected to the barrel jack port through a cable rated at 5V to ensure the board did not receive power above what it can handle. At the same time, the UART bridge is now powered solely by the OBD-II connection, so this portion of the project has reached the **stretch goal**.

2. UI Design

For the UI component the team aimed high even for commitment. The team estimated that VGA output would be able to display pure RPM data at the least. This was quickly proved not to be the case upon design, and the team has not figured out this solution up to this point.

The VGA output does however display static formatting for the data to be plotted onto, including graphs for each respective set of data, ASCII character labels and the outline for average values off to the side. This portion proved more difficult than imagined, and thus the team **did not meet** the initial commitment benchmark.

3. Latency Processing

Since the design is communication with the OBD-II port via UART connection, there is always going to be a degree of latency. When deciding the goals for this section the team kept this in mind and planned accordingly.

The low latency of the Nexys A7-100T FPGA stems from its parallel hardware architecture, direct access to internal BRAM, and the absence of software layers or operating system overhead. Data is processed in real time using dedicated logic blocks and FSMs, with immediate response to UART signals and rapid memory access — often within a single clock cycle. When interfacing with a car's ECU, however, the latency

bottleneck typically arises on the ECU's side. This is because the ECU must interpret the incoming OBD-II request, retrieve sensor data, and generate a response over UART — a process that may involve internal software routines, bus arbitration, and queued data handling. As a result, while the FPGA can process incoming data nearly instantaneously, the delay between sending a request and receiving a response is largely dictated by the ECU's internal architecture and timing behavior.

Once the team figured out the clock cycle and how to initiate this process it was easy to ensure the low latency, meaning once again the **stretch goal** has been achieved.

4. ECU Interaction

The FPGA interaction with the automobile's ECU is a pivotal part to the project, as nothing that follows this process would be possible if codes are not sent and data is not received. The FPGA is able to send out the necessary codes and receive the requested values. Upon reading the data through a logic analyzer the team could see clear and correct information communicated back and forth.

The extra hardware used to communicate with the ECU of the car was the ELM327, an integrated circuit that allows users that generalizes PID's to a set list of commands. The team also had to utilize a bi-directional logic level shifter since the output of the ELM27 is a 5 volt output which means that the output voltage was too high for the PMOD input of the FPGA. Using the level shifter the team was able to safely convert a 5 volt logical signal to a safe 3.3 volt signal. Beyond that the team also used a L705CV 5 volt voltage regulator to provide 5V Vcc into the level shifter from a 9V D battery.

Because both the read and write functions and data were so clear the team determined this qualified as the **stretch goal** for the ECU interaction.

5. Product Design

Lastly, the team defined multiple levels of housing for the components ranging from loose wiring and the product sprawled out to neatly stowed away in a 3D printed container with outlets to connect all required cables and outside components.

At this time the FPGA board and extending cables and wires are stored in a tote, providing a compact storage option. The team is currently working on a 3D printed design that, as described, encases all necessary components and adds a professional feel to the design. For the time being, the design is at the **goal** checkpoint with the possibility to reach the stretch goal in the near future.

2. High-Level Description (with key terminology)

This project implements a real-time visualization system that bridges a vehicle's **OBD-II (On-Board Diagnostics II)** interface with a **VGA (Video Graphics Array)** display using a **Field-Programmable Gate Array (FPGA)**. At the core of the design is a custom **UART (Universal Asynchronous Receiver/Transmitter)** controller responsible for sending **PID (Parameter ID)** request codes to the vehicle's **Engine Control Unit (ECU)** and receiving periodic data, such as engine RPM or coolant temperature.

To handle the difference in clock speeds between the 153.6 kHz UART or a **9600 Baud Rate** interface and the 100 MHz FPGA system clock, the design uses **FIFO (First-In, First-Out)** buffers to manage **clock domain crossing** and maintain stable communication. The incoming data is parsed and decoded into simplified 16-bit hexadecimal values and stored in the FPGA's internal **BRAM (Block RAM)** for real-time access.

Graphical output is generated by a VGA pipeline that includes a **frame buffer** and **loader module**, which format and display the data as dynamic line graphs. Each frame includes axes, a black background for contrast, and **ASCII**-rendered average values beside each graph for clarity. Graph points are plotted in real time, advancing horizontally as new values are written to memory.

The system is organized into discrete, testable modules controlled by multiple **Finite State Machines (FSMs)**. These manage the flow of data through transmission, reception, memory storage, and VGA rendering stages. This modular architecture not only enhances system reliability but also makes the design replicable and adaptable for a variety of real-time diagnostic applications.

3. Main Modules, Finite State Machines (FSMs)

The team's design is built from several modular components written in Verilog, each controlled by a finite state machine (FSM) to manage how data flows from the car's ECU to the VGA display. These FSMs handle tasks such as sending and receiving UART messages, storing sensor data, and updating the graphical display in real time. Together, they create a responsive, low-latency system suitable for reading and visualizing automotive diagnostics data. The main top module pseudo flowchart can be seen seem below in **Figure 3.0.1**

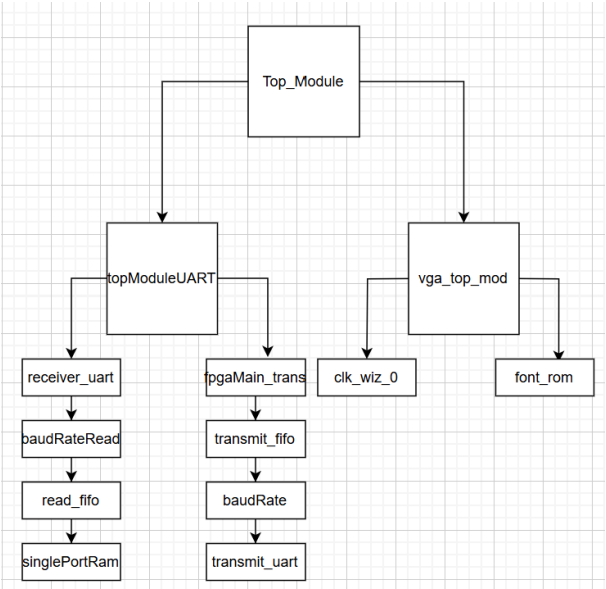

Figure 3.0.1: Flowchart of Project

Transmit (UART)

UART serial communication utilizes two primary data lines: a transmit (TX) line and a receive (RX) line on each device. These lines normally form somewhat of an X shape during wiring due to the fact that the TX of one device connects to the RX of the other. In our implementation, the TX line from the FPGA, routed through PMOD port JB[2], is connected to the RX input of the ELM327 interface.

The transmit path of the UART system is composed of three core modules: the Transmit Controller, the Transmit FIFO, and the Transmitter. Each module is responsible for a distinct stage of the communication process, enabling reliable and continuous data exchange between the FPGA and the ECU.

Transmission begins with the Transmit Controller, which is responsible for orchestrating communication sequences. This module is built around a finite state machine (FSM) that governs the sending of predefined commands to the ECU. The state machine transitions through a series of stages, including initialization, data polling, and command repetition, ensuring continuous communication throughout system operation. The controller buffers outgoing commands by loading them into the Transmit FIFO for subsequent transmission. The list of ECU commands generated by the controller, including both startup sequences and ongoing data requests, is summarized in **Figure 3.1** below. **Figure 3.1.1** shows the basic FSM for the controller module.

PID Action	**ASCII Equivalent**
Initialize ELM327	ATZ/r
Disable Echo	ATE0/r
Find Protocol	ATSP/r
Initialize connection with ECU	0100/r
Request Engine RPM	101C/r
Request Coolant Temperature	0105/r

Figure 3.1 Table of relevant PID's

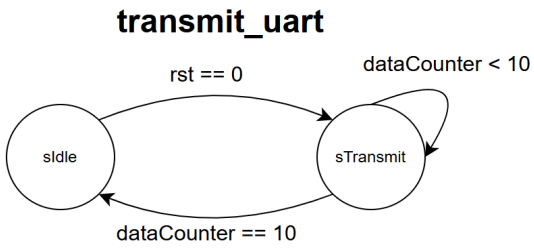

Figure 3.1.1: Transmit module FSM

The next stage of the transmit process involves synchronization between the high-speed Transmit Controller and the slower moving UART Transmitter module. This synchronization is achieved through a First-In-First-Out (FIFO) data structure. What this module is acting as is essentially a buffer. Operating at the 100 MHz system clock, the Controller is capable of preparing and loading data rapidly, whereas the Transmitter, limited by the 156.3 kHz baud rate clock (corresponding to 9600 baud), processes outgoing bytes at a significantly slower rate.

The FIFO serves as a critical intermediary, decoupling the timing between these two modules. It operates based on a handshake signaling protocol: the Controller asserts a valid data signal when new information is available, and the FIFO acknowledges once the data has been successfully stored and made ready for transmission. This mechanism ensures reliable communication across clock domains, prevents data loss, and allows the Controller to continue loading subsequent commands without needing to stall or synchronize directly with the UART transmission timing. The FSM for the FIFO can be seen below in **Figure 3.1.2**

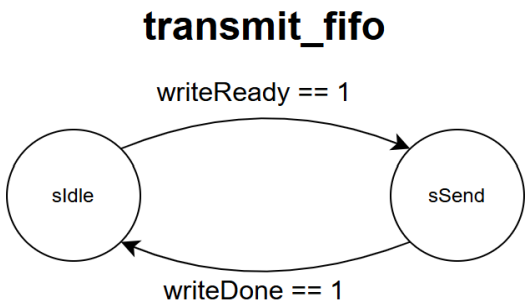

Figure 3.1.2: Transmit FIFO FSM

The final stage of the transmit process is the UART Transmitter module. Functionally, the Transmitter is responsible for serializing the buffered data for transmission over the UART interface. It accepts an 8-bit parallel input from the FIFO and frames it into a 10-bit UART packet by appending a start bit at the beginning and a stop bit at the end, following standard UART protocol conventions.

The serialized data is then transmitted bit-by-bit at a frequency of 156.3 kHz, corresponding to a baud rate of 9600. Upon completing the transmission of a byte, the Transmitter asserts a signal back to the FIFO, requesting the next byte to be loaded into its data register. This handshaking mechanism ensures continuous and collision-free data flow between the FIFO and the Transmitter while maintaining synchronization with the slower UART clock domain. The FSM for the Controller can be seen below in **Figure 3.1.3**:

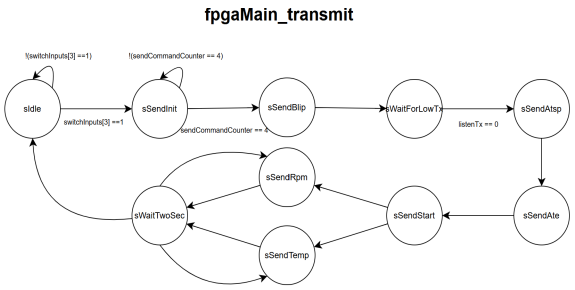

Figure 3.1.3: Transmit Controller FSM

Receive(UART)

The receive side of the UART communication process is composed of two primary modules: the Receiver and the FIFO/Processing Unit. Unlike the transmit path, where the Controller initializes data flow, the receive process begins with the receiver module accepting incoming data from the ELM327. The Receiver module operates similarly to the Transmitter, functioning in the 156.3 kHz clock domain, consistent with the 9600 baud UART standard. It continuously checks the incoming signal for a falling edge on the ELM327 TX line, which indicates the start bit of a UART packet. Upon detecting this transition, the Receiver captures the subsequent 10 bits — one start bit, eight data bits, and one stop bit. After assembling a complete frame, the Receiver extracts the eight valid data bits and forwards them to the FIFO/Processing Unit .**Figure 3.2.1** shows the FSM for the receiver.

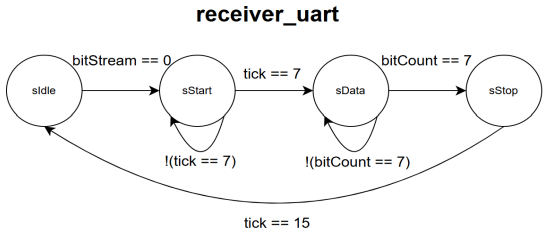

Figure 3.2.1: Receiver FSM

Due to synthesis constraints encountered during development, the FIFO in the receive path serves a dual role: both buffering incoming bytes and directly interfacing with the RAM storage subsystem. After

receiving a new byte, the Receiver asserts a handshaking signal to the FIFO, similar to the mechanism used in the transmit path, indicating that a new data word is ready for storage or processing.

The final stage of the UART receive path involves the FIFO and RAM insertion module. After the FIFO receives a handshaking signal from the Receiver indicating valid data availability, it captures the incoming byte and initiates the data parsing process. The received data arrives in binary format, representing ASCII-encoded hexadecimal values. Consequently, an ASCII-to-decimal conversion must be performed before the information can be correctly interpreted and stored.

The data parsing process involves reading pairs of ASCII characters, converting them into their corresponding hexadecimal and then decimal values, and extracting the meaningful parameters from the decoded payload. Within the scope of this project, the system specifically targets and processes responses for Engine RPM (PID 010C) and Engine Coolant Temperature (PID 0105) requests.

A sample ECU response and the associated decoding steps are illustrated in **Figure 3.2**. Additionally, **Figure 3.3** provides the transfer functions used to translate the received ASCII data into human-readable values for RPM and temperature. **Figure 3.3.1** shows the FIFO FSM.

Type of Response	**ASCII Response**
Engine RPM	41 0C 1A F8>
Coolant Temperature	41 05 5A>

Figure 3.2: Table for Relevant Responses

Measurement	**Response Bytes**	**Transfer function**
RPM	2 Bytes	RPM = ((A * 256) + B) / 4
Coolant Temp	1 Byte	(°C) = A - 40

Figure 3.3: Transfer Functions for Relevant Responses

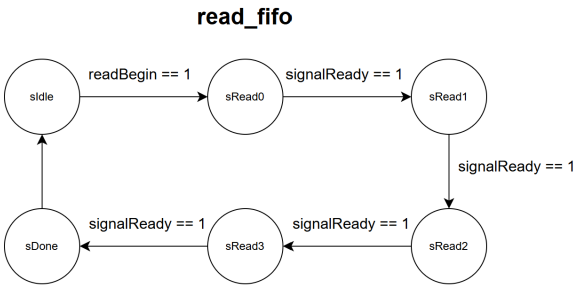

Figure 3.3.1: FSM for Read FIFO

Block Ram

The block RAM interface (singlePortRam.v) serves as the primary storage buffer for incoming ECU data. This module includes an FSM with idle and load states that continuously writes values to memory when writeEnable is asserted. It also provides a read port for accessing specific data locations. Visual status indicators (greenLed, redLed) update based on whether the memory is still being written or full.

The averaging module (readRam.v) is responsible for computing the average of a block of stored values, typically used to smooth out noise in sensor readings. It uses a state machine to loop through memory addresses, accumulate values into a register, and divide the result once all samples are read. This helps produce more stable visual output on the VGA display.

Baud Rate Generation

The baud rate generator (baudRate.v) and sampling synchronizer (baudRateRead.v) generate precise timing pulses for UART communication. baudRate.v uses a simple FSM to toggle a baud pulse at a programmable interval derived from the 100 MHz system clock, while baudRateRead.v extends this with pulse-length modulation to produce a pulse that lasts for multiple clock cycles—making UART sampling more reliable and deterministic. The read baud rate generator is shown below in **Figure 3.4** and the write baud rate generator is shown in **Figure 3.5**

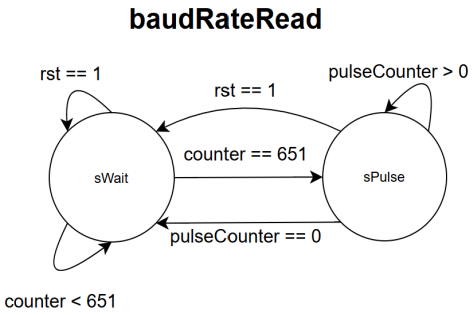

Figure 3.4: Read Baud Rate Generator FSM

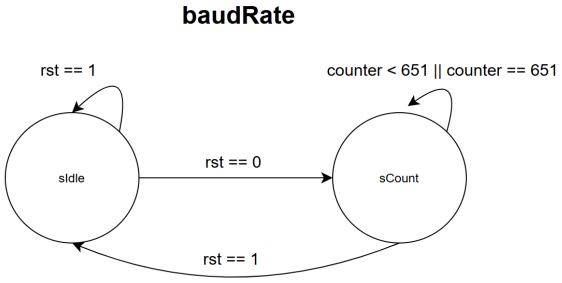

Figure 3.5: Write Baud Rate Generator FSM

VGA

The VGA resolution used for this project is 1920 by 1080, or 1080p for short. To achieve this there must be a clock signal of 148.5 MHz, faster than the internal 100 MHz clock of the FPGA board. The Vivado clock wizard (clk_wiz_0.xci) produced a file that was able to alter the 100 MHz clock to 145.8 MHz. This clocking wizard uses a mixed-mode clock manager to achieve this The higher resolution allows the team to plot more precise data when the live readings are being received from the onboard BRAM.

There is a single module (font_rom.v) that handles all of the ASCII characters used to label each graph and display the outputs of average coolant temperature and engine rpm.Each necessary character is identified by its unique ASCII value, and then pretend in an eight by eight array of pixels. For non-alphabetical characters the module only defines the rows of pixels that are necessary for that character to appear.

Design Evaluation

1. Latency and Throughput, Overhead

This section focuses primarily on the latency aspects of communication between the ECU and the FPGA. Due to the use of UART as the communication protocol, the system inherently experiences higher latency compared to other serial communication standards. UART operates asynchronously with no shared clock line, with both devices internally clocked at 156.3 kHz, corresponding to 9600 baud. Data transfer is thus inherently limited to this speed. Additional latency is introduced by the ECU itself, which must fetch the response data and reply again over the 9600 baud rate. In practice, this results in a latency of approximately 200 milliseconds from one response to another, which aligns with expectations based on the baud rate and typical ECU response times. While acceptable for this project, it reinforces that UART is best suited for short, non-time-critical data exchanges.

In terms of throughput, the UART controller processes one byte of data at a time, including framing with a start and stop bit for each transaction. The design omitted parity bit handling, as the ELM327 and ECU communication did not require it, simplifying the controller logic. Overheads introduced by the design included both hardware and software considerations. On the hardware side, a level-shifter circuit was incorporated to safely convert the ELM327's 5V UART output to the 3.3V PMOD voltage levels acceptable by the FPGA. This prevented potential damage to the FPGA while maintaining reliable data transmission. On the software side, error detection control logic was implemented in the read modules. Specifically, the logic scanned incoming data streams for known response headers, such as the 41 0C identifier for the Engine RPM PID response. If the expected header was not detected, the module discarded the entire response packet and awaited the next transmission, ensuring system robustness against corrupted or malformed packets.

2. Block RAM (BRAM)

The use of memory in this project centered around a single-port block RAM module designed to store temperature response data collected from the ECU. Given that temperature PID requests were sent once per second during operation, a five-minute drive would require approximately 300 entries. Each entry occupied 16 bits of storage, leading to a total requirement of 4,800 bits. To account for potential overhead and ensure

robustness, the block RAM was provisioned with 500 entries, totaling 8,000 bits (~8 Kb). This memory footprint was easily accommodated within the available BRAM resources on the FPGA, requiring only a small fraction of the total available memory blocks.

The development and integration process for the single-port RAM module was straightforward. However, initial synthesis attempts encountered an issue where Vivado optimized the block RAM away as unused logic. To address this, a simple initialization state machine was implemented to write zeros across all memory locations during startup, ensuring Vivado recognized the RAM as an active and utilized structure.

Given the current project scope, the allocated RAM is more than sufficient. Future revisions could expand memory usage to simultaneously log both temperature and engine RPM data during a five-minute drive, effectively doubling the array size. Another potential expansion could allow for data collection across longer drives, such as ten or fifteen minutes, which would require a proportional increase in memory capacity. In early discussions, the team considered using the most significant bit (MSB) of each memory word as a simple flag to distinguish between temperature and RPM values, allowing efficient parsing of mixed data types without significant overhead. The FSM for the ram generator is shown below in **Figure 3.6**:

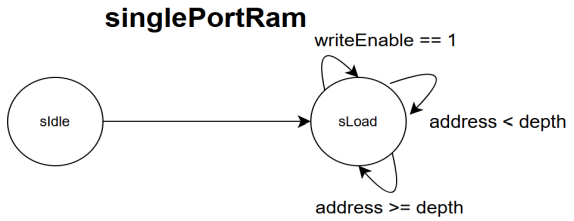

Figure 3.6 Single Port Ram FSM

3. Timing Requirements

Multiple areas of the design require different clock speeds to function. The ODB-II to UART connection requires a clock speed of 153.6 kHz, the equivalent of 9600 baud.

The VGA resolution decided on was 1920x1080, or 1080p. This requires a 60 Hz refresh rate, along with a 148.5 MHz pixel clock. The Vivado clock wizard (clk_wiz_0) was useful in creating a clock signal that altered the 100 MHz system clock into the desired 148.5 MHz clock for output.

The font for VGA display (font_rom.v) handles all of the ASCII characters, both numbers and symbols, that display as graph labels and above average telemetry data off to the side of each graph.

4. Use Cases

The design is specific to cars that include OBD-II protocol as means to communicate with the ECU, which covers a majority of cars on the road. The only two sets of data this revision supports are engine RPM and coolant temperature. The overall design meets the team's stretch goals. This does not mean all project requirements were fulfilled but the team believes this was more than they expected to complete.

Design Testing

The design testing process adopted a hierarchical approach to verification. Each module was individually validated through simulation before being progressively integrated into larger subsystems. On the serial communication side, both the UART transmit and receive modules were extensively tested using standalone testbenches, culminating in final system-level testbenches such as **sendAtzTransmit.v** for transmission and **tb_fullRead.v** for reception. Similarly, the VGA controller and block RAM modules were tested independently before full system integration. A key aspect of the testing methodology was the implementation of self-checking features in the important testbenches, which automated error detection and eliminated the need for manual waveform inspection. This approach significantly improved debugging efficiency during development.

Randomized testing was intentionally avoided. For UART modules, packetized data structures demanded precise, known input sequences to validate proper functionality, making randomization counterproductive. Although deterministic inputs were less critical for VGA and RAM modules, the consistent non-random testing methodology was maintained across all modules to promote repeatability and traceability during integration. Simulation results confirmed correct behavior across all modules, with no errors detected by the self-checking mechanisms. All simulations were conducted using Vivado's simulator, with selected cross-validation performed using iVerilog.

To complement the written results, a diagram illustrating the hierarchical test flow (individual module test → subsystem integration test → full system validation) and selected waveform screenshots from the UART self-checking testbenches will be included to provide a visual summary of the design validation process.

It is important to note that all the waveforms for all the team's testbenches can be found in the Section 3 of the Appendix.

Implementation Insights

1. FIFO-Based Clock Domain Crossing

One of the most significant insights was the use of FIFO buffers to bridge the timing gap between the 153.6 kHz UART clock and the 100 MHz system clock. The team initially attempted to directly connect the UART interface to internal logic, which led to data instability and synchronization issues. By inserting dual-clock FIFO modules (read_fifo.v, transmit_fifo.v), the system could safely transfer data across timing domains. This change not only stabilized communication but also simplified the design of the control FSMs.

2. Custom Block RAM

Rather than using IP-generated memory, the team developed a custom single-port BRAM module (singlePortRam.v) to store incoming sensor data. This allowed for more control over memory depth, data width, and integration with other hand-written modules. Building BRAM manually also made the memory access logic easier to test and simulate alongside other parts of the system. This decision supported the project's educational goal of maximizing hardware design experience. In future versions the team will implement the Vivado IP catalog to generate BRAM.

3. Onboard Switches for Mode Selection

The team used the Nexys A7 board's onboard switches as a simple and reliable user interface for selecting sensor data (e.g., RPM or temperature) and initiating data requests. This choice avoided the need for additional I/O or external buttons, reducing potential sources of error and making the system easier to use in a real-world context.

4. Clock Generation via Clocking Wizard IP

To meet the strict timing requirements of 1920×1080 VGA output, the team used **Vivado's Clocking Wizard IP** to generate a precise 148.5 MHz clock signal derived from the FPGA's 100 MHz onboard oscillator. This clock was used exclusively for the VGA timing engine, ensuring accurate sync pulse generation and proper pixel scanning. Using the Clocking Wizard simplified the clock setup while maintaining signal precision and stability across the design.

5. Hardware UART Debugging

To verify communication with the ECU, the team connected the UART transmit and receive lines to a logic analyzer. This allowed direct observation of byte-level data transfers and helped confirm the accuracy of initialization sequences and PID responses. This hardware validation step was essential in confirming reliable UART performance outside of simulation. A sample response packet from the Logic Analyzer is shown in Section 5 of the appendix.:

Source Code Link

The link attached below directs the reader to the team's GitHub that is composed of all the necessary files for project design and implementation.

UART Controller Code

https://github.com/abhibogga/CSEE4280Workspace/tree/main/final_Project

VGA Controller Code:

https://github.com/abhibogga/CSEE4280Workspace/tree/main/final_Project/verliog%20work

---

Video Link

The link below shows all our progress in a demonstration:

https://youtu.be/p_PrYKqfLGs?si=2mTZtMrYP8s3n_ri

Appendix

Power Design	This covers how we power the UART module as well as the FPGA within this design	FPGA is powered by laptop and UART module is powered off additional DC supply	FPGA is powered by laptop and UART module finds internal power source, no external power source for UART mod	FPGA is powered either by car ECU or external battery, UART module is powered by OBD-II
UI Design	This will cover how the user sees and digests information via the monitor	Monitor displays pure rev data, non-formmatted	Monitor displays formmatted data such as average temp and rpm data, as well as either a graph of rpm data or temp data	Monitor displays formmatted data such as average temp and rpm data, as well as a graph of rpm data and temp data. All data should be shown with explanations to user
Latency Processing	Since the serial communcation is UART, there will always be some sort of latency, but this project can attempt to do some latency design	Project reads information with ECU and FPGA latency	Project reads information with only ECU latency, FPGA works fast to avoid any latency on its end	Project reads information with only ECU latency, FPGA works fast to avoid any latency on its end
ECU Interaction	This will cover how the project will communciate with the cars computer	FPGA can only read information, errors in transmission are expected	FPGA can only read information, no errors in transmission	FPGA not only can read information but can clear and write information as well
Product Design	This will be how the FPGA and controllers are stored	All wires are connected but loosely stored	Wires and FPGA board are stashed in box so product feels complete	Wires, FPGA, and UART module are stored away in 3D Printed box with outlets for DB9 cable

FIgure 1.1.1: Project Checklist

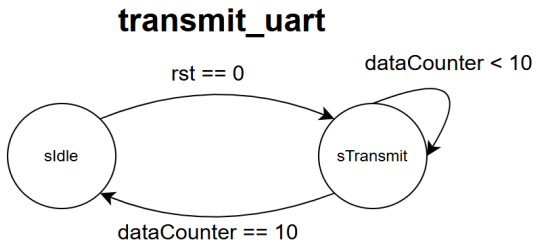

Figure 1.3.1: transmit_uart FSM

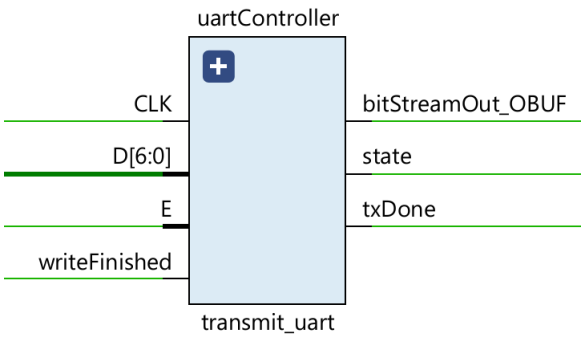

Figure 1.3.2: transmit_uart Block Diagram

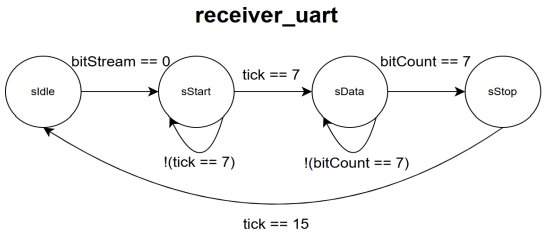

Figure 1.3.3: receiver_uart FSM

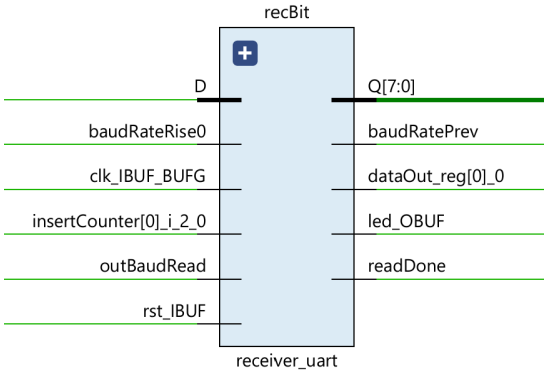

Figure 1.3.4: receiver_uart Block Diagram

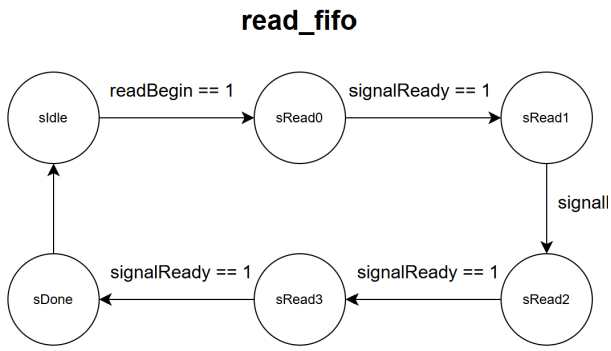

Figure 1.3.5: read_fifo FSM

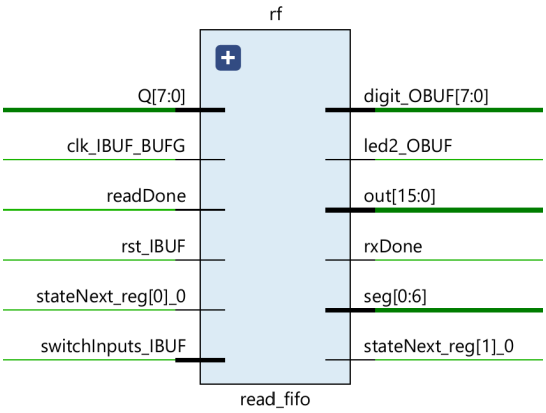

Figure 1.3.6: read_fifo Block Diagram

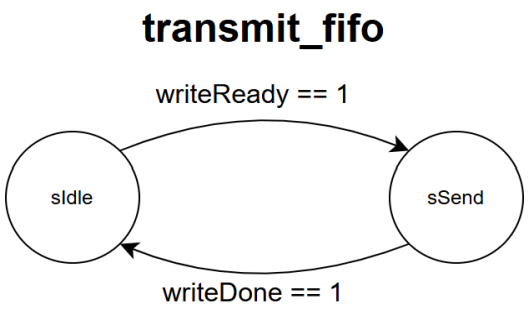

Figure 1.3.7: transmit_fifo FSM

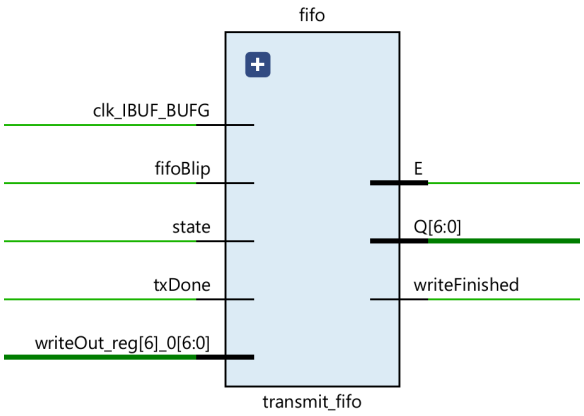

Figure 1.3.8: transmit_fifo Block Diagram

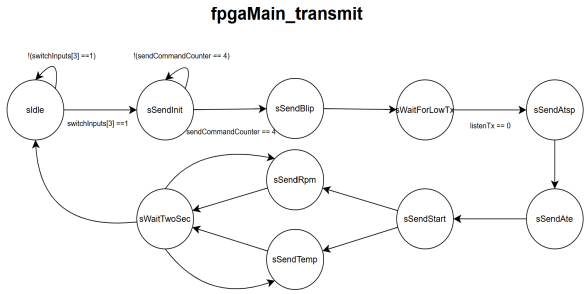

Figure 1.3.9: fpgaMain_transmit FSM

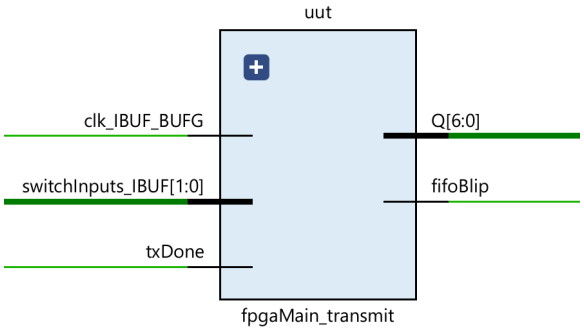

Figure 1.3.10: fpgaMain_transmit Block Diagram

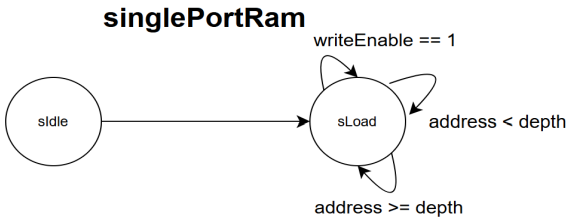

Figure 1.3.11: singlePortRam FSM

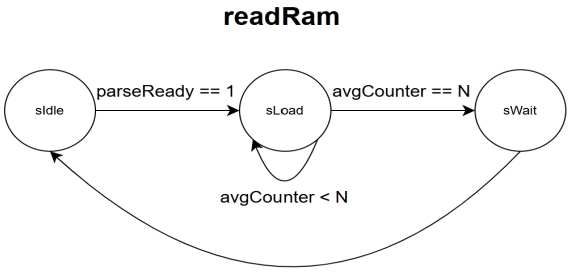

Figure 1.3.13: readRam FSM

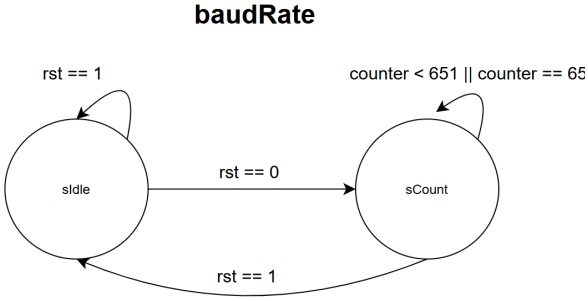

Figure 1.3.14: baudRate FSM

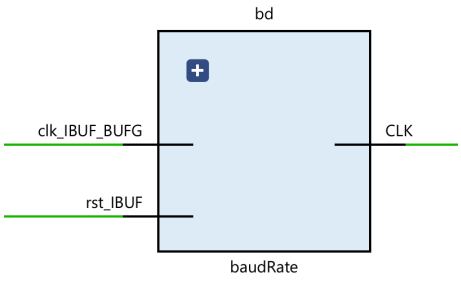

Figure 1.3.15: baudRate Block Diagram

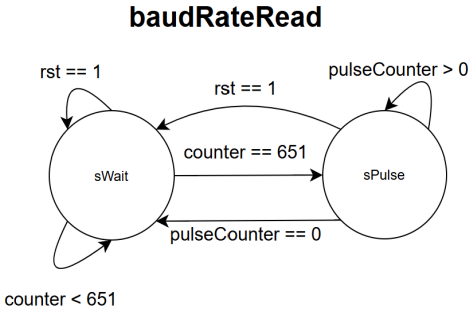

Figure 1.3.16: baudRateRead FSM

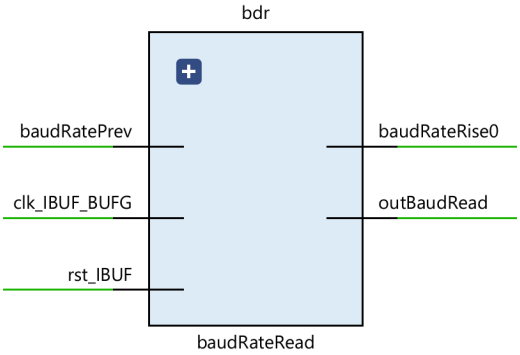

Figure 1.3.17: baudRateRead Block Diagram

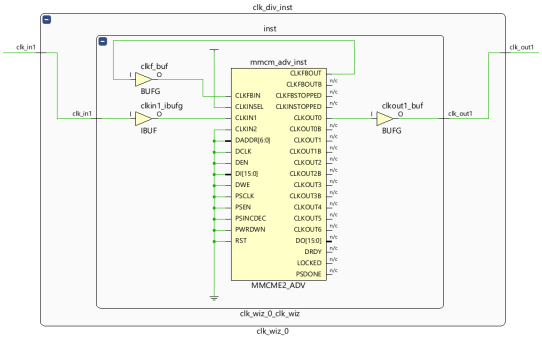

Figure 1.3.18: clk_wiz_0 Block Diagram

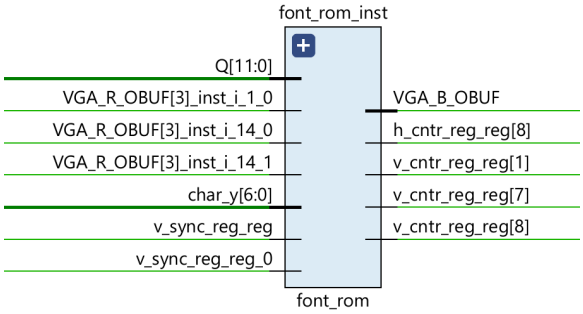

Figure 1.3.19: font_rom Block Diagram

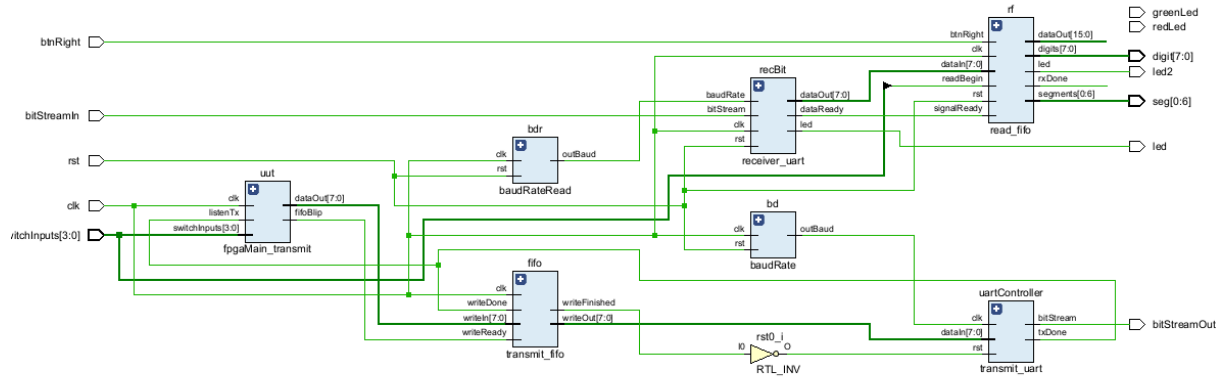

Figure2.1: Total UART controller Block Diagram

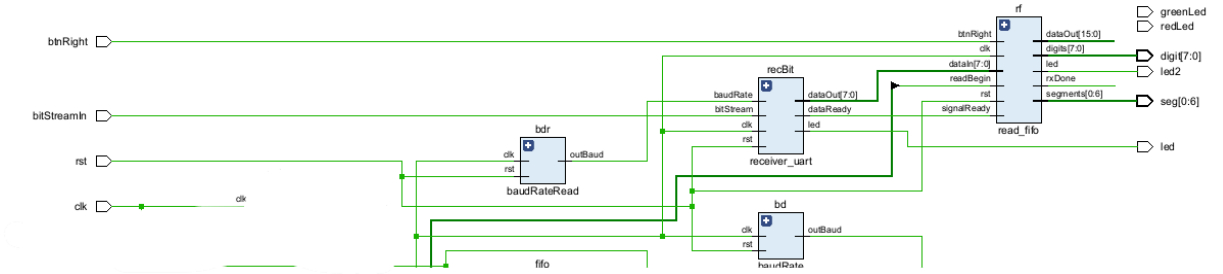

Figure 2.2: Complete UART Read Block Diagram

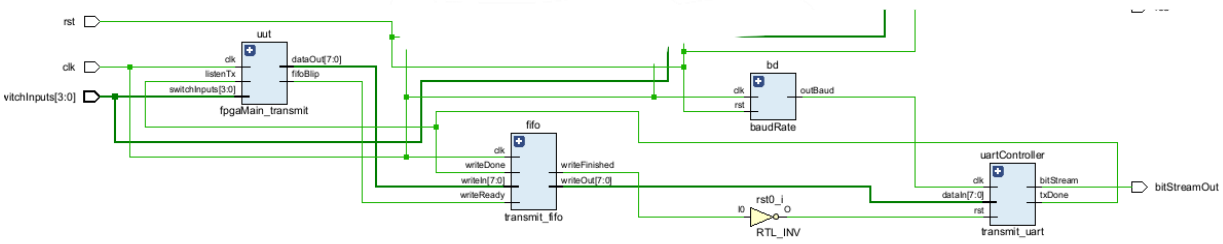

Figure 2.3: Complete UART Write Block Diagram

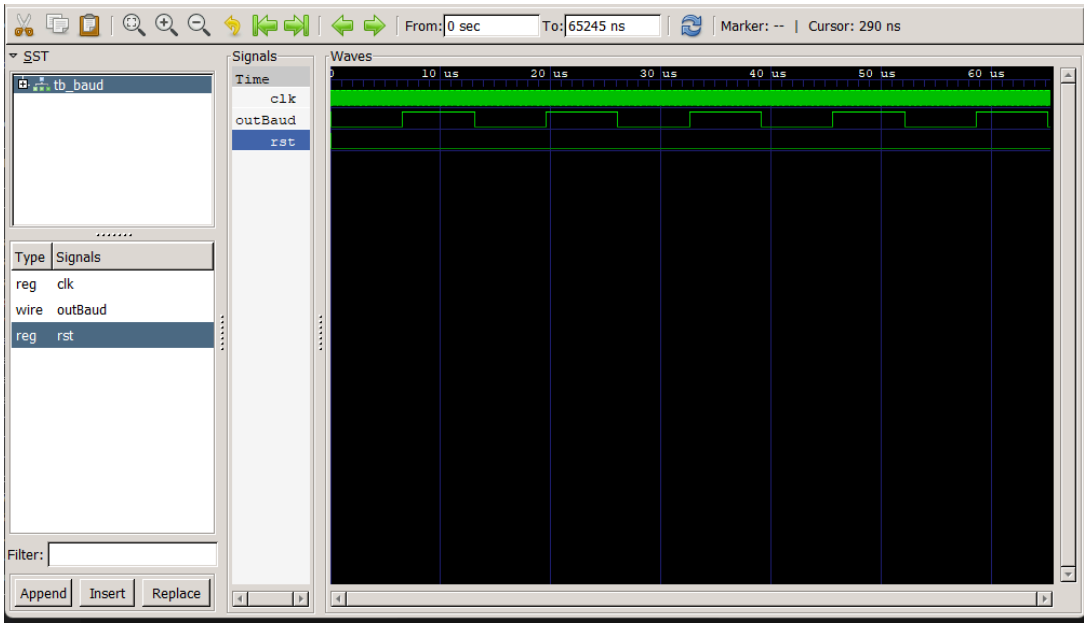

Figure 3.1: buadRate_tb

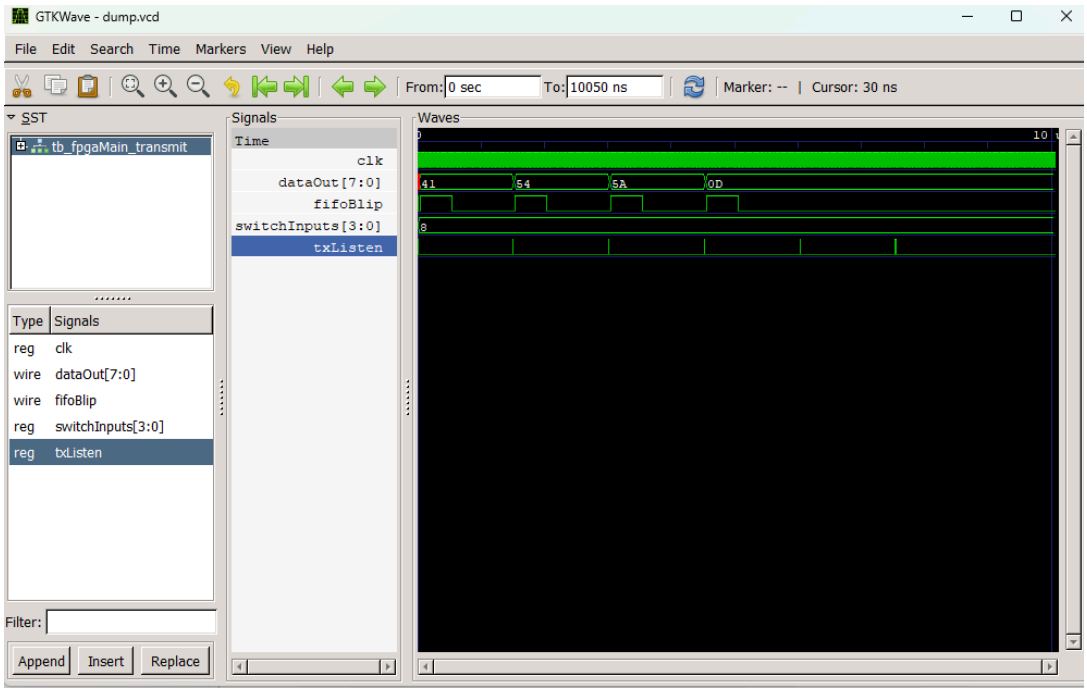

Figure 3.2: fpgaMain_transmit

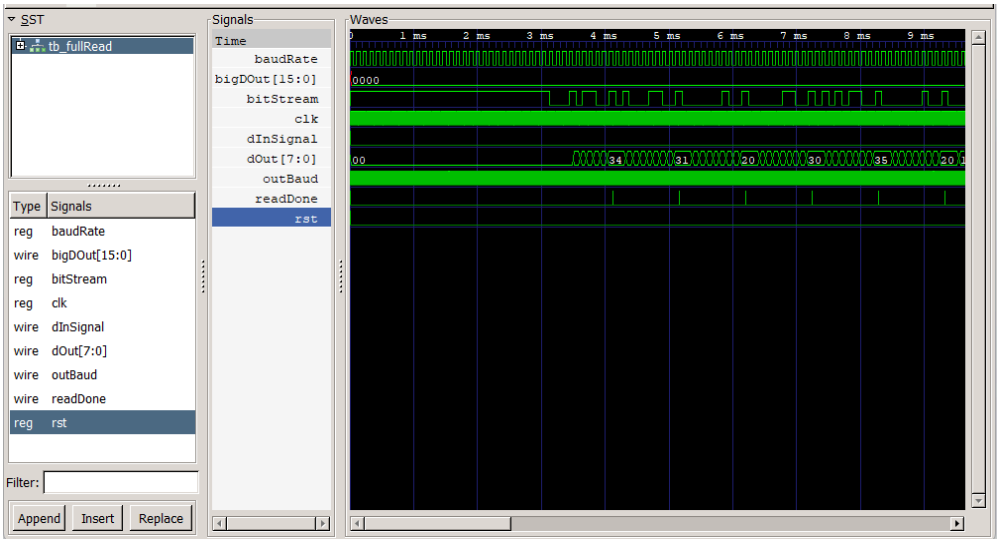

Figure 3.3: tb full read

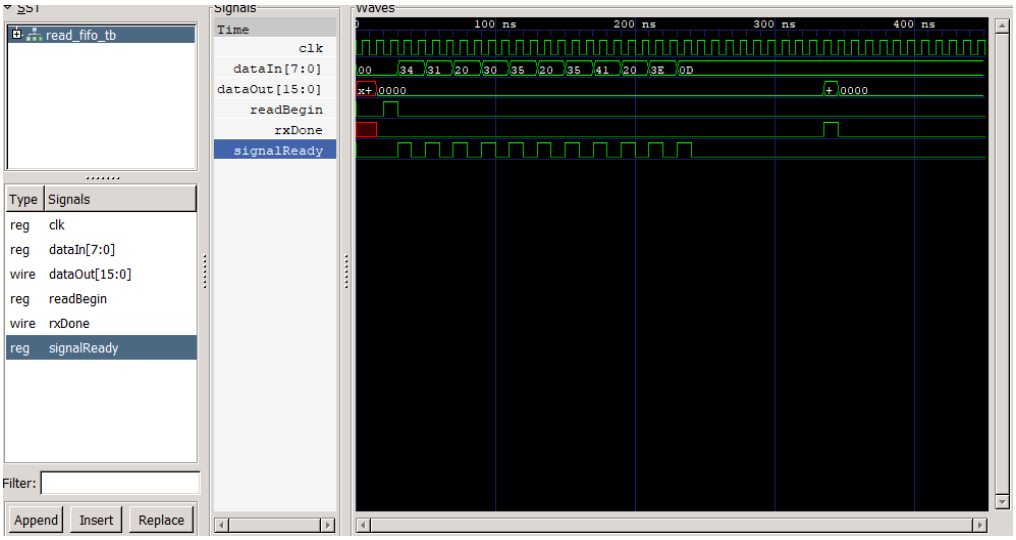

Figure 3.4: read fifo

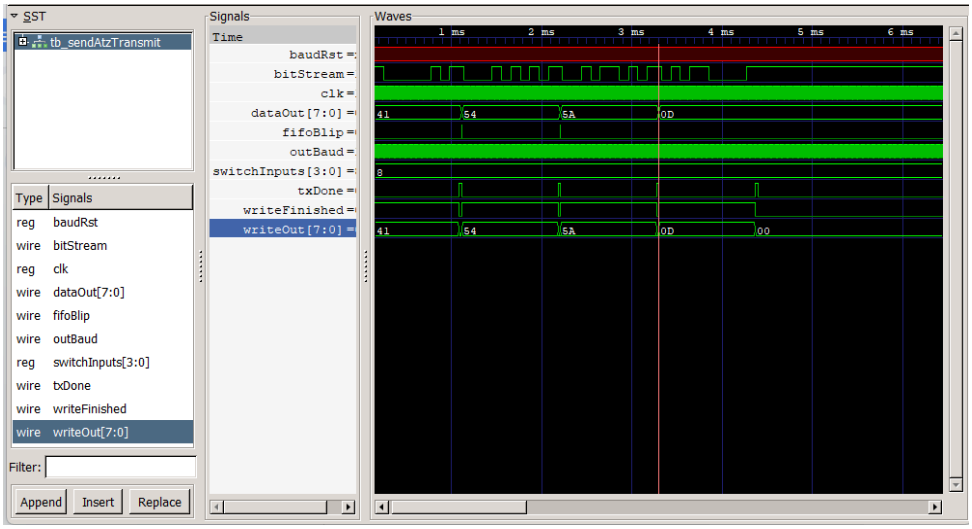

Figure 3.5: sendAtzTransmit (transmitController top)

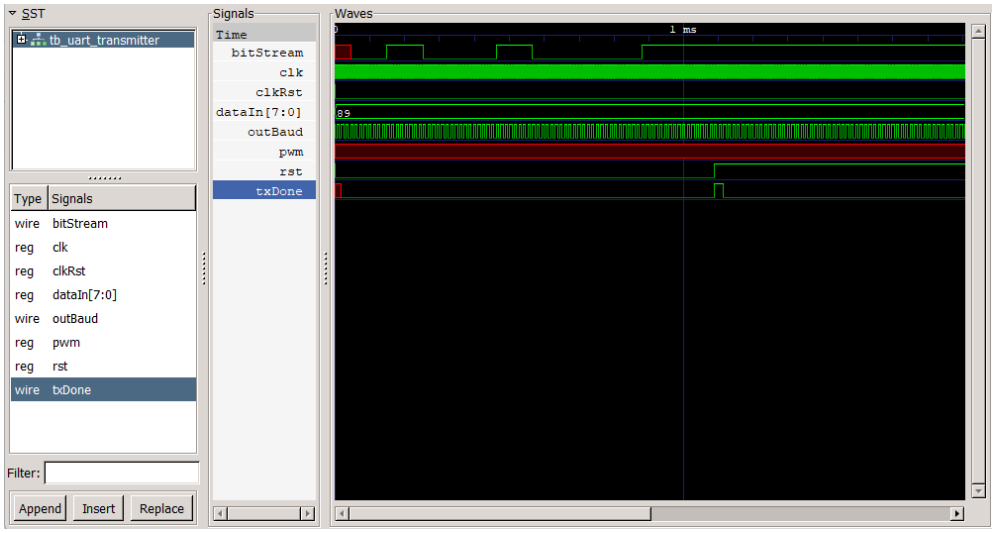

Figure 3.6: uart_tranmitter

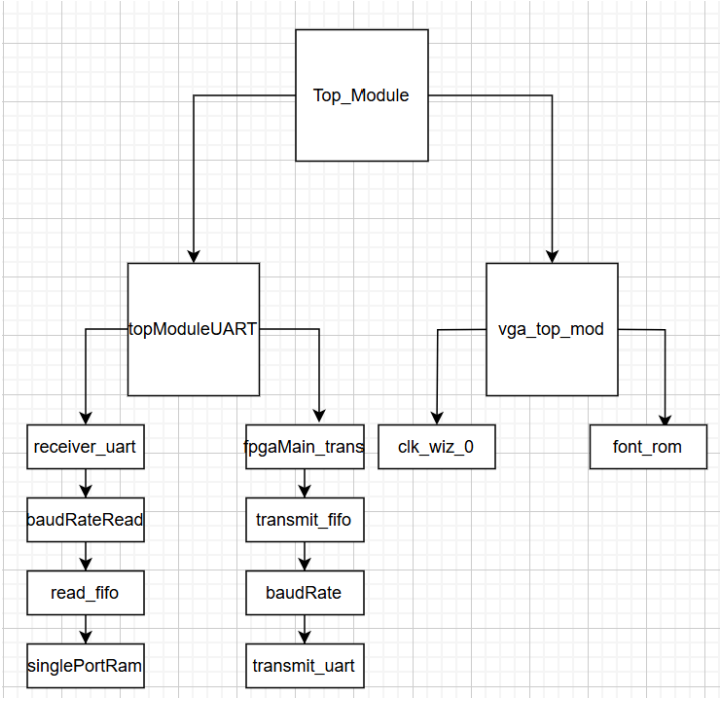

Figure 4.1: Hierarchy of Ideal Design

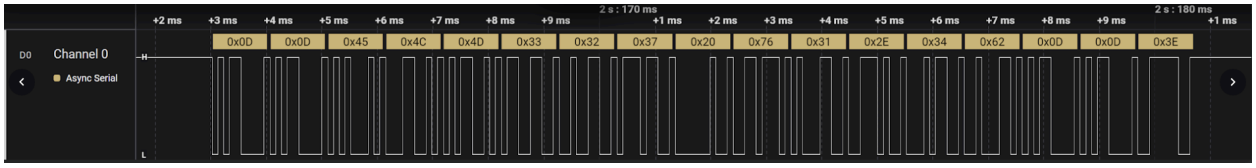

FIgure 5.1: Logic Analyzer Output ELM Initialization

References

[1] DashLogic, OBD-II J1979 PID listing, https://www.dashlogic.com/docs/technical/obdii_pids (accessed Apr. 25, 2025).

[2] Digilent, Nexys A7TM FPGA Board Reference Manual, https://digilent.com/reference/_media/reference/programmable-logic/nexys-a7/nexys-a7_rm.pdf (accessed Apr. 25, 2025).

[3] V-Codes, "How to use AMD Vivado's IP Catalog to create a Block RAM," YouTube, https://www.youtube.com/watch?v=Ag0ogQJbkSY (accessed Apr. 25, 2025).

[4] Digilent, "Digilent/arty-pmod-VGA," GitHub, https://github.com/Digilent/Arty-Pmod-VGA (accessed Apr. 25, 2025).

[5] TONI_K, "OBD II UART Hookup Guide," OBD II UART Hookup Guide - SparkFun Learn, https://learn.sparkfun.com/tutorials/obd-ii-uart-hookup-guide/all#:~:text=Once%20you%20have%20your%20headers,prompt%20in%20the%20terminal%20window. (accessed Apr. 25, 2025).

[6] FPGA Discovery, YouTube, https://www.youtube.com/watch?v=L1D5rBwGTwY (accessed Apr. 25, 2025).

[7] Russell, "UART in VHDL and Verilog for an FPGA," Nandland, https://nandland.com/uart-serial-port-module/ (accessed Apr. 25, 2025).

[8] TONI_K, "Getting started with OBD-II," Getting Started with OBD-II - SparkFun Learn, https://learn.sparkfun.com/tutorials/getting-started-with-obd-ii (accessed Apr. 25, 2025).

[9] Elm327 OBD to RS232 interpreter, https://cdn.sparkfun.com/assets/learn_tutorials/8/3/ELM327DS.pdf (accessed Apr. 26, 2025).

[10] metroschultz, "Any interest in developing an open source fuel economy gauge/computer? - page 9 - fuel economy, hypermiling, EcoModding News and forum," EcoModder, https://ecomodder.com/forum/showthread.php/any-interest-developing-open-source-fuel-economy-gauge-1428-9.html (accessed Apr. 25, 2025).

[11] mviljoen2 and Instructables, "Hack an ELM327 cable to make an Arduino OBD2 scanner," Instructables, https://www.instructables.com/Hack-an-ELM327-Cable-to-make-an-Arduino-OBD2-Scann/ (accessed Apr. 25, 2025).